Chapter 11
Pipeline Architecture Style

Pipeline Architecture Style

Introduction

- One of the fundamental styles in software architecture that repeatedly appears is the pipeline architecture (also known as the pipes and filters architecture).
- This pattern followed as soon as developers and architects decided to split functionality into discrete parts.
- Most developers know this architecture as the underlying principle behind Unix terminal shell languages, such as Bash and Zsh.
- Developers in many functional programming languages will see parallels between language constructs and elements of this architecture. Many tools that utilize the MapReduce programming model follow this basic topology. While these examples show a low-level implementation of the pipeline architecture style, it can also be used for higher-level business applications.

Pipeline Architecture Style

Topology (1)

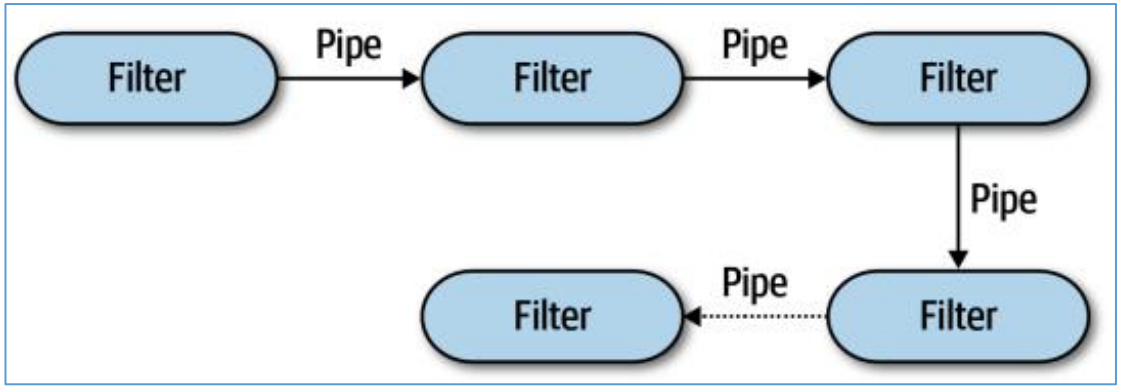
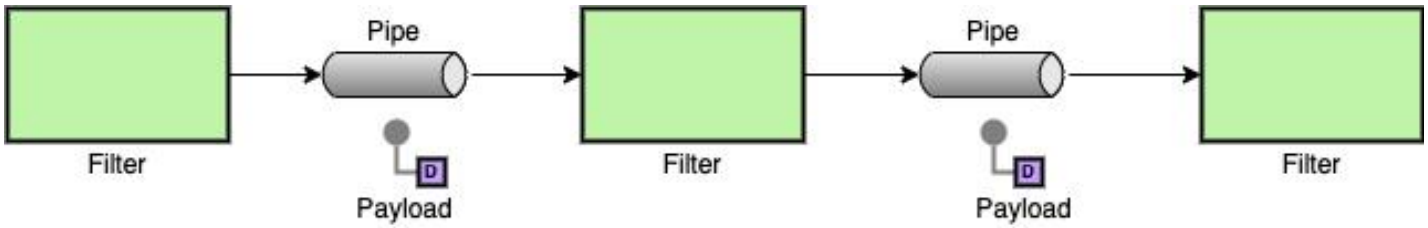


Figure 11-1. Basic topology for pipeline architecture

- Pipes
  - Pipes in this architecture form the communication channel between filters. Each pipe is typically unidirectional and point-to-point (rather than broadcast) for performance reasons, accepting input from one source and always directing output to another. The payload carried on the pipes may be any data format, but architects favor smaller amounts of data to enable high performance.



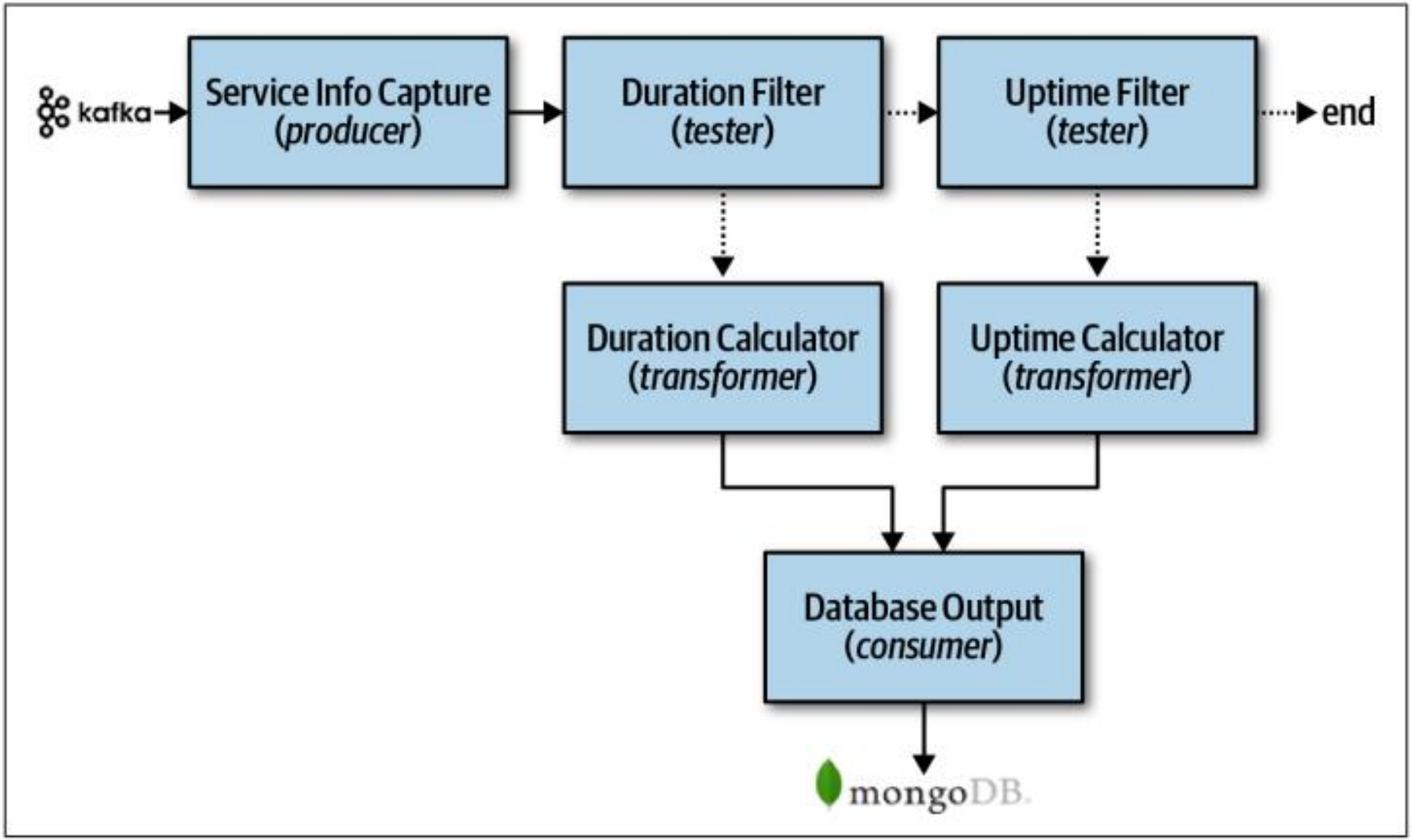
Pipeline Architecture Style

Topology (2) - Filters

- Filters are self-contained, independent from other filters, and generally stateless. Filters should perform one task only. Composite tasks should be handled by a sequence of filters rather than a single one. Four types of filters exist within this architecture style:
- Producer
  - The starting point of a process, outbound only, sometimes called the source.
- Transformer
  - Accepts input, optionally performs a transformation on some or all of the data, then forwards it to the outbound pipe. Functional advocates will recognize this feature as map.
- Tester
  - Accepts input, tests one or more criteria, then optionally produces output, based on the test. Functional programmers will recognize this as similar to reduce.
- Consumer
  - The termination point for the pipeline flow. Consumers sometimes persist the final result of the pipeline process to a database, or they may display the final results on a user interface screen

Pipeline Architecture Style

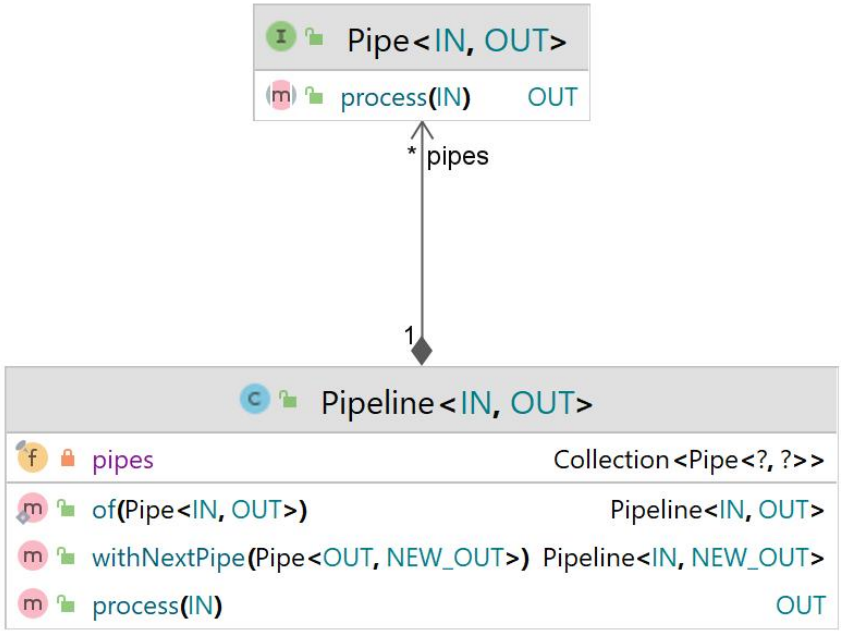
Example



Pipeline Architecture Style

Example 1

```
public interface Pipe<IN, OUT> {
    OUT process(IN input);
}
```




```
public static void main(String[] args) {
    Pipeline<Integer, Long> pipeline =
            Pipeline.of(input ->
input.longValue() * 2);
    pipeline = pipeline
            .withNextPipe(input -> input + 4)
            .withNextPipe(input -> input + 2)
            .withNextPipe(input -> input + 5);
    Long result = pipeline.process(5);
    System.out.println(result);
}
```

Pipeline Architecture Style

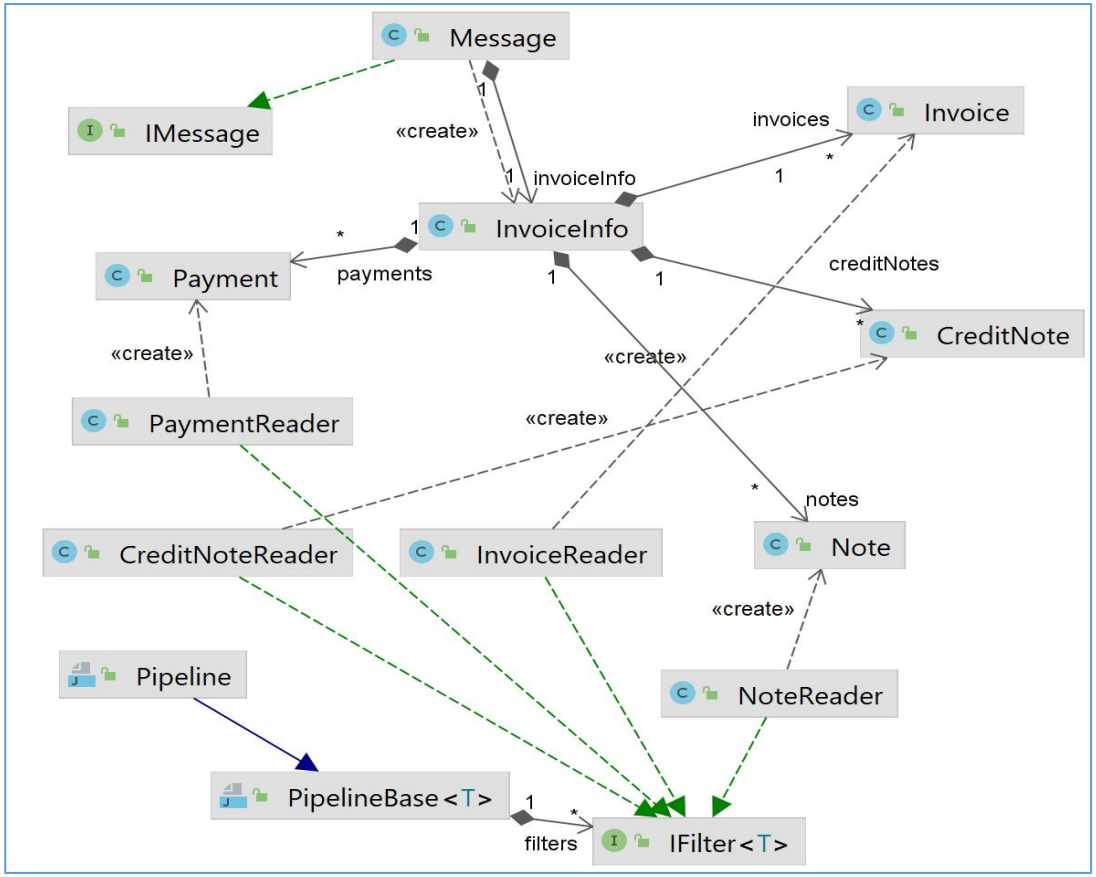
Example 2 – using Function library

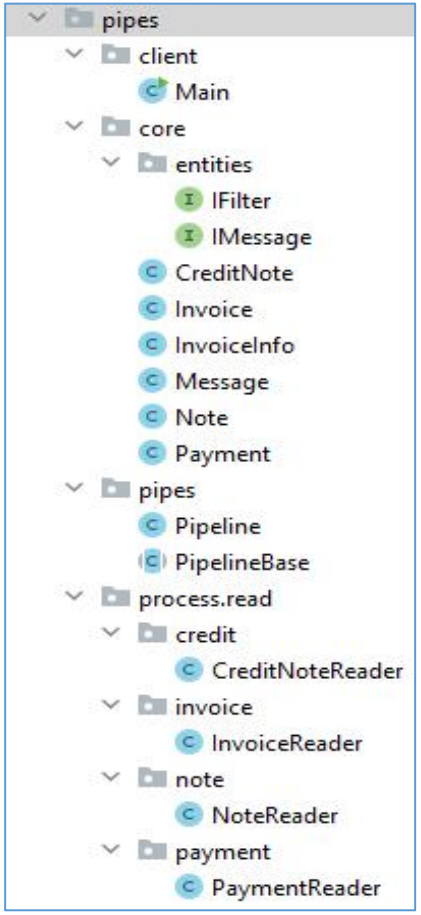
```
public interface Pipe<IN, OUT> {
    OUT process(IN input);

    default <NEW_OUT> Pipe<IN, NEW_OUT> add(Pipe<OUT, NEW_OUT> pipe) {
        return input -> pipe.process(process(input));
    }
}
```

```
import java.util.function.Function;

public class Main {
    public static void main(String[] args) {
        Function<Integer, Integer> square = s -> s * s;
        Function<Integer, Integer> half = s -> s / 2;
        Function<Integer, String> toString =
Object::toString;
        Function<Integer, String> pipeline =
square.andThen(half)
                .andThen(toString);
        String result = pipeline.apply(5);
        System.out.println(result);
    }
}
```

Pipeline Architecture Style

Example 3





Pipeline Architecture Style

Architecture Characteristics Ratings

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★★
Elasticity	★
Evolutionary	★★★
Fault tolerance	★
Modularity	★★★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★★